

CSE 256 | Lecture 6: Learning Word Representations

Ndapa Nakashole

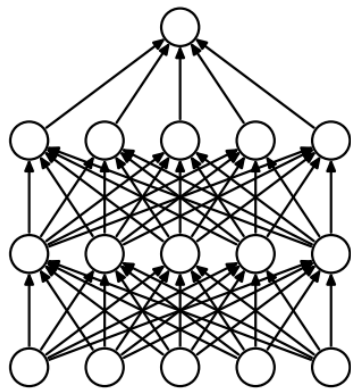
January 22, 2026

Recap: Feedforward Neural Networks

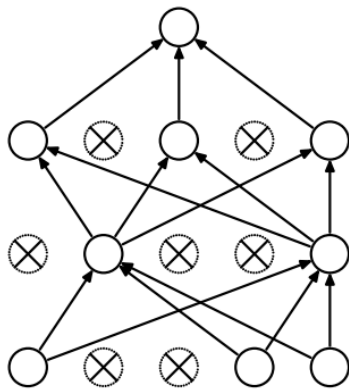
A deep neural network is obtained by **repeating the same computation**:

$$\begin{aligned}h^{(1)} &= g\left(W^{(1)}x + b^{(1)}\right) \\h^{(2)} &= g\left(W^{(2)}h^{(1)} + b^{(2)}\right) \\&\vdots \\h^{(L)} &= g\left(W^{(L)}h^{(L-1)} + b^{(L)}\right)\end{aligned}$$

Recap: Dropout Regularization



(a) Standard Neural Net

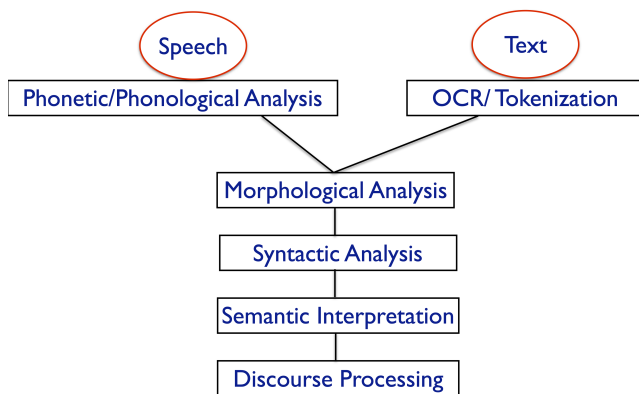


(b) After applying dropout.

A Shift Toward **Language** Data

- ▶ So far in the course:
 - ◉ Core machine learning tools
 - ◉ Linear models and neural networks
- ▶ Starting today:
 - ◉ Applying these tools to **language**
 - ◉ We will start thinking more about **language data** and its unique properties

Levels of Language Analysis



Today

- ➊ Motivation for Word Representations
- ➋ Distributional Semantics
- ➌ Learning Word Embeddings (a foundational idea in NLP)
 - Word2Vec (Skip-gram)
 - GloVe
- ➍ Evaluating Word Embeddings
- ➎ Tokenization: What Counts as a Word?

How to Represent Meaning in NLP Systems?

- ▶ **Goal of semantics in NLP:** Accurately encode the meaning of **words**, **sentences**, and **documents** inside computers
- ▶ Challenge:
 - ◉ Meaning is intuitive for humans
 - ◉ But must be made **explicit and computable** for machines

The Traditional Answer: WordNet

Meaning is encoded explicitly as a graph of concepts

- ▶ For many years, NLP represented meaning using **symbolic lexical resources**
- ▶ **WordNet**: a large, manually curated lexical database
 - ◉ Words grouped into **synsets** (sets of synonyms)
 - ◉ Explicit semantic relations between words
- ▶ Examples of relations encoded in WordNet:
 - ◉ **Synonymy**: *car* $\leftrightarrow$ *automobile*
 - ◉ **Hypernymy (is-a)**: *mouse* $\rightarrow$ *rodent* $\rightarrow$ *animal*
 - ◉ **Hyponymy**: *animal* $\rightarrow$ *dog*, *cat*

WordNet Example

```
In [2]: from nltk.corpus import wordnet as wn
mouse = wn.synset('mouse.n.01')
hyper = lambda s: s.hypernyms()
list(mouse.closure(hyper))
```

```
Out[2]: [Synset('rodent.n.01'),
         Synset('placental.n.01'),
         Synset('mammal.n.01'),
         Synset('vertebrate.n.01'),
         Synset('chordate.n.01'),
         Synset('animal.n.01'),
         Synset('organism.n.01'),
         Synset('living_thing.n.01'),
         Synset('whole.n.02'),
         Synset('object.n.01'),
         Synset('physical_entity.n.01'),
         Synset('entity.n.01')]
```

Problems with hand-crafted resources like WordNet

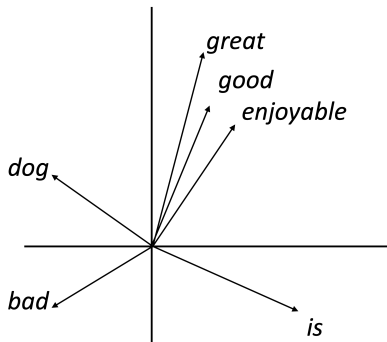
- ▶ **Limited nuance in similarity**
 - ◉ Are these really interchangeable?
 - ◉ *adept, expert, good, practiced, proficient, skillful*
- ▶ **Hard to keep up with language change**
 - ◉ New or informal words:
 - ◉ *wicked, badass, nifty, ace, ninja, genius*
- ▶ **Manually constructed**
 - ◉ Requires human labor
 - ◉ Subjective and slow to adapt to new domains
- ▶ **Not designed for graded similarity**
 - ◉ Difficult to compute continuous word similarity
 - ◉ No natural notion of “how close” two words are

Word Embeddings

Idea: Word meaning can be represented by vectors of real numbers.

Want: Words with **similar meaning** should have **similar vectors**.

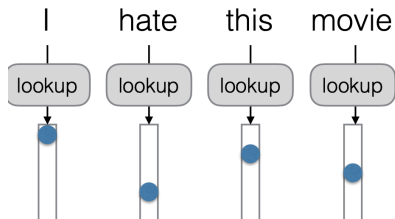
the movie was great
 $\approx$
the movie was good



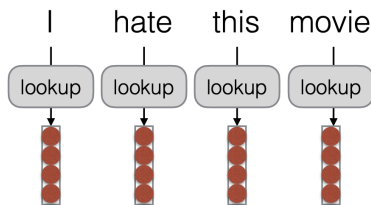
- ▶ We want to **learn** these vectors from data
- ▶ Dense, low-dimensional vectors (e.g., 100-300 dimensions) vs 1-hot (very high-dimensional, sparse vectors)

Sparse vs Dense Representations

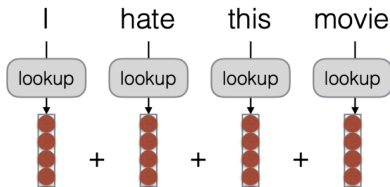
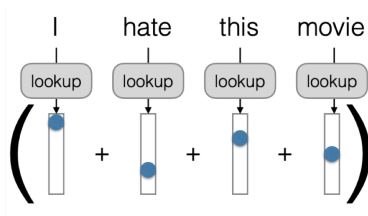
One-hot Representations



Dense Representations



Bag of Words vs CBOW



Conceptual foundation for word embeddings:

Distributional Semantics

Learning Word Embeddings via Distributional Semantics

We will use the idea of **distributional semantics** to learn word embeddings.

kombucha

I had a glass of kombucha at the farmers market

Children or pregnant women should not drink kombucha

- ▶ **Distributional semantics:** words that appear in similar contexts have similar meanings
 - ◉ We could substitute “kombucha” with “beer” or “wine” in these sentences
- ▶ **Observe** many usages of a word and infer its meaning from context

Word Embeddings

- ▶ Each word is represented by a **dense vector** of real numbers
- ▶ Vectors are learned so that words appearing in similar contexts have **similar vectors**
 - ⊙ Similarity is measured using the **dot product**

$$\textit{banking} = \begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \end{pmatrix}$$

$$\textit{monetary} = \begin{pmatrix} 0.413 \\ 0.582 \\ -0.007 \\ 0.247 \\ 0.216 \\ -0.718 \\ 0.147 \\ 0.051 \end{pmatrix}$$

(Mikolov et al., 2013): Word2Vec

Word2Vec (Mikolov et al., 2013)

- ▶ **Word2Vec**: a family of models for learning word embeddings that are **simple** and **efficient**
- ▶ Two main algorithms:
 - ◉ **Skip-gram** and **CBOW**
 - ◉ The two are closely related; we will focus on **Skip-gram**

Skip-gram: Learning Word Embeddings (Mikolov et al., 2013)

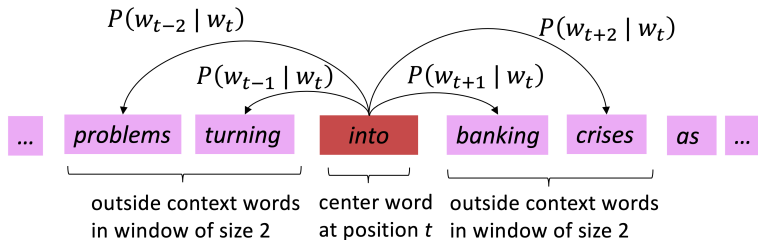
- ▶ **Input:** a large text corpus
- ▶ **Output:** a dense vector (embedding) for each word in the vocabulary
- ▶ **Learning:** learn embeddings by solving a *proxy prediction task*
 - ◉ Given a center word, predict each of its surrounding context words

Example: Center Word = “into”

Predict Context Words

Context window size = 2

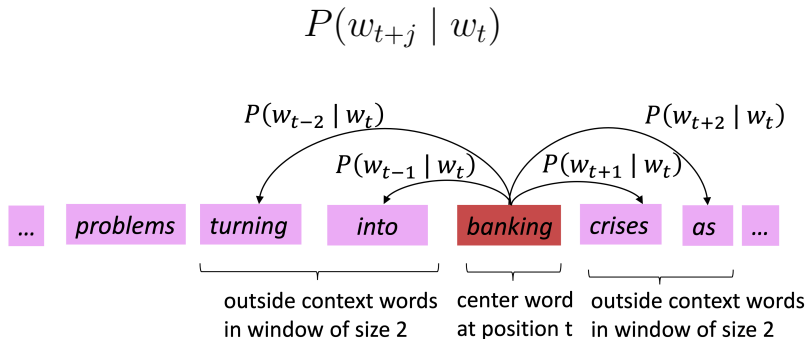
$$P(w_{t+j} \mid w_t)$$



Example: Center Word = “banking”

Predict Context Words

Context window size = 2



- ▶ This process is repeated for **every word position** in the corpus
- ▶ The word embeddings are learned as a byproduct of this prediction task

Skip-gram Loss Function

For each position $t = 1, \dots, T$, we predict the surrounding **context words** within a window of size m , given the **center word** w_t .

Data Likelihood:

$$\prod_{t=1}^T \prod_{\substack{-m \leq j \leq m \\ j \neq 0}} P(w_{t+j} \mid w_t; \theta)$$

where θ denotes all model parameters (i.e., the word embeddings).

Training Loss (Negative Log-Likelihood):

$$L(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} \mid w_t; \theta)$$

Skip-gram Loss Function (continued)

We want to minimize the loss:

$$L(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} \mid w_t; \theta)$$

Question: How do we compute

$$P(w_{t+j} \mid w_t; \theta) ?$$

Answer: Using a **softmax** over the vocabulary (**linear model**):

$$P(o \mid c) = \frac{\exp(u_o^\top v_c)}{\sum_{w \in V} \exp(u_w^\top v_c)}$$

with center word c and a context word o :

We use **two vectors per word** w :

- ▶ v_w : when w is a **center** word
- ▶ u_w : when w is a **context** word

Training Word Vectors: Gradient Descent

- ▶ We learn word vectors by **minimizing the Skip-gram loss** using gradient descent
- ▶ θ denotes **all model parameters**:
 - ◉ input (center) word vectors v_w
 - ◉ output (context) word vectors u_w
- ▶ Each word has **two vectors** during training (works out better for optimization)

$$\theta = \begin{bmatrix} v_{aardvark} \\ v_a \\ \vdots \\ v_{zebra} \\ u_{aardvark} \\ u_a \\ \vdots \\ u_{zebra} \end{bmatrix} \in \mathbb{R}^{2dV}$$

Computing the Gradients (High-Level)

The Skip-gram loss is:

$$L(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} \mid w_t; \theta)$$

The probability term is a **softmax**:

$$P(o \mid c) = \frac{\exp(u_o^\top v_c)}{\sum_{w \in V} \exp(u_w^\top v_c)}$$

To train the model, we compute gradients such as:

$$\frac{\partial}{\partial v_c} \log \frac{\exp(u_o^\top v_c)}{\sum_{w \in V} \exp(u_w^\top v_c)}$$

Computing the Gradients w.r.t. the Center Word Vector

We start with the log-softmax term:

$$\frac{\partial}{\partial v_c} \log \frac{\exp(u_o^\top v_c)}{\sum_{w=1}^V \exp(u_w^\top v_c)}$$

Using log rules, we can rewrite this as:

$$\frac{\partial}{\partial v_c} \left(u_o^\top v_c - \log \sum_{w=1}^V \exp(u_w^\top v_c) \right)$$

Step 1: Differentiate the first term

$$\frac{\partial}{\partial v_c} (u_o^\top v_c) = u_o$$

Gradient w.r.t. the Center Word Vector (2/4)

Step 2: Differentiate the normalization term

$$\log \sum_{w=1}^V \exp(u_w^\top v_c)$$

Using the chain rule ($\frac{d}{dz} \log z = \frac{1}{z}$):

$$\frac{\partial}{\partial v_c} \log \sum_{w=1}^V \exp(u_w^\top v_c) = \frac{1}{\sum_{w=1}^V \exp(u_w^\top v_c)} \frac{\partial}{\partial v_c} \sum_{w=1}^V \exp(u_w^\top v_c)$$

Gradient w.r.t. the Center Word Vector (3/4)

Step 2: Differentiate the normalization term (continued)

$$\frac{\partial}{\partial v_c} \sum_{w=1}^V \exp(u_w^\top v_c)$$

Move the derivative inside the sum:

$$\begin{aligned} \frac{\partial}{\partial v_c} \sum_{w=1}^V \exp(u_w^\top v_c) &= \sum_{w=1}^V \frac{\partial}{\partial v_c} \exp(u_w^\top v_c) \\ &= \sum_{w=1}^V \exp(u_w^\top v_c) u_w \end{aligned}$$

Gradient w.r.t. the Center Word Vector (4/4)

Putting it all together

$$\frac{\partial}{\partial v_c} \log P(o | c) = u_o - \frac{1}{\sum_{w=1}^V \exp(u_w^\top v_c)} \sum_{x=1}^V \exp(u_x^\top v_c) u_x$$

negative log likelihood gradient:

$$\begin{aligned} &= -u_o + \sum_{x=1}^V \frac{\exp(u_x^\top v_c)}{\sum_{w=1}^V \exp(u_w^\top v_c)} u_x \\ &= -u_o + \sum_{x=1}^V P(x | c) u_x \end{aligned}$$

Gradient descent update:

$$v_c \leftarrow v_c + (\text{true context vector} - \text{model's prediction})$$

Gradient w.r.t. the Context Word Vector: Home Exercise

- ▶ The gradient with respect to a context vector u_o has a form **analogous** to the gradient w.r.t. the center vector
- ▶ It also has an **observed minus expected** structure
- ▶ **At home:** derive the gradient w.r.t. the context word vector as an exercise

Why gradients rarely vanish in Skip-gram

A gradient is said to **vanish** when its magnitude becomes so small that parameter updates are negligible.

- ▶ Softmax rarely assigns probability exactly 1 to any word in skipgram
- ▶ Words appear in many different contexts
- ▶ Embeddings must balance multiple, sometimes conflicting contexts
- ▶ Training uses stochastic, noisy gradient estimates

Embeddings keep receiving small corrective updates throughout training.

Hyperparameters in Word Embeddings

- ▶ Dimensionality: 50-300 dimensions
- ▶ Window size: 5-10 words
- ▶ Initialization of word embeddings
- ▶ Learning rate
- ▶ Training epochs

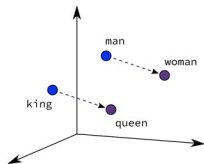
Are my word embeddings any good?

Intrinsic Evaluation: Word Clustering

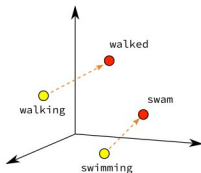


Are my word embeddings any good?

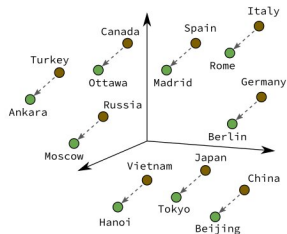
Intrinsic Evaluation: Analogies



Male-Female



Verb Tense



Country-Capital

a:b :: c:?



$$d = \arg \max_i \frac{(x_b - x_a + x_c)^T x_i}{\|x_b - x_a + x_c\|}$$

man:woman :: king:?

Are my word embeddings any good?

Extrinsic Evaluation: Downstream Task Performance

- ▶ **Approach 1:** Learn embeddings from scratch as parameters from your data: Often works pretty well
- ▶ **Approach 2:** Initialize using pretrained word embeddings, keep fixed. Faster because no need to update these parameters
- ▶ **Approach 3:** initialize using pretrained word embeddings, fine-tune on task. Often works best

Word2Vec: Capturing cooccurrences inefficiently

- ▶ Go through each word of the whole corpus
- ▶ Predict surrounding words of each (window's center) word
- ▶ This captures cooccurrence of words **one at a time**
- ▶ **Glove**: why not capture cooccurrence counts directly?

GloVe: Global Vectors for Word Representation: Pennington et al. (2014)

Example Corpus:

I like deep learning .

I like NLP .

I enjoy flying .

counts	I	like	enjoy	deep	learning	NLP	flying	.
I	0	2	1	0	0	0	0	0
like	2	0	0	1	0	1	0	0
enjoy	1	0	0	0	0	0	1	0
deep	0	1	0	0	1	0	0	0
learning	0	0	0	1	0	0	0	1
NLP	0	1	0	0	0	0	0	1
flying	0	0	1	0	0	0	0	1
.	0	0	0	0	1	1	1	0

GloVe: Global Vectors for Word Representation” (Pennington et al. (2014)

Glove loss function:

$$L(\theta) = \frac{1}{2} \sum_{i,j=1}^{\mathcal{V}} f(X_{ij}) (u_i^T v_j - \log X_{ij})^2$$

- ▶ $\mathcal{V}$: vocabulary size
- ▶ X_{ij} : cooccurrence count of word i and j
- ▶ u_i, v_j : word vectors
- ▶ $f(X_{ij})$: weighting function to reduce the influence of very frequent or very rare word pairs

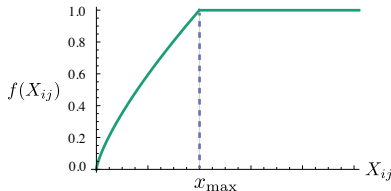


Figure: $f(X_{ij})$

What counts as a word?

Tokenization

What Is Tokenization?

- ▶ A **token** is the basic processing unit used by NLP models
- ▶ **Tokenization** is the process of converting raw text into a sequence of tokens

Typical pipeline:

Text → Tokens → Token IDs

Example (simple word-level tokenization):

The quick brown fox jumps over the lazy dog

11 298 34 567 432 13 11 49 305

Tokenizer

- ▶ A **tokenizer** is a separate module, independent of the model
- ▶ It is trained (or designed) using its **own data and rules**
- ▶ The tokenizer acts as a **translation layer** between raw text and numerical representations

Tokenizer interface:

- ▶ **Encode**: raw text $\rightarrow$ token IDs
- ▶ **Decode**: token IDs $\rightarrow$ raw text

Tokenization: A Source of Model Behavior and Limitations

- ▶ Tokenization shapes what the model can easily represent and learn
- ▶ Examples:
 - ◉ **Programming languages (e.g., Python):** token boundaries affect how code structure and whitespace are represented
 - ◉ **Numbers and math:** splitting numbers into multiple tokens can lead to brittle numerical reasoning
 - ◉ **Punctuation and formatting:** contractions, hyphens, and symbols introduce ambiguity

Problems with Whitespace Tokenization

Where whitespace tokenization breaks:

- ▶ prize-winning $\Rightarrow$ prize, -, winning
- ▶ 1850s $\Rightarrow$ 1850, s
- ▶ U.K. $\Rightarrow$ U, ., K
- ▶ 2.5 $\Rightarrow$ 2, ., 5

Takeaway: Word-level tokenization requires many **special-case rules** for punctuation, numbers, contractions, hyphens, etc.

- ▶ Comprehensive tokenizers (e.g., spaCy) implement hundreds of such rules

GPT3 Tokenizer: struggles with programming languages

<https://platform.openai.com/tokenizer> : try it out!

Observation: Early LLM tokenizers struggled with programming languages, especially Python, splits spaces and indentation inefficiently

Tokens	Characters
161	331

```
Tokenization is at the heart of much weirdness of LLMs. Do
not brush it off.
127 + 677 = 804
1275 + 6773 = 8048
Egg.
I have an Egg.
egg.
EGG.
for i in range(1, 101):
    if i% 3== i% and i = \
    print("FizzBuzz")
    elif i % 3 == 0:
    print("Fizz")
    elif i % 5 == 0:
    print("Buzz")
    else:
```

GPT3.5/4 Tokenizer improves Python code handling

Better tokenization = denser representations = more usable context.

- ▶ Groups more white space into a single token, densifies Python code, can attend to **more code in one forward pass**
- ▶ **Takeaway:** Tokenizer design directly affects what models can learn.

Tokens	Characters
120	331

```
Tokenization is at the heart of much weirdness of LLMs. Do
not brush it off.
127 + 677 = 804
1275 + 6773 = 8048
Egg.
I have an Egg.
egg.
EGG.
for i in range(1, 101):
    if i % 3 == 0 and i % 5 != 0:
        print("FizzBuzz")
    elif i % 3 == 0:
        print("Fizz")
    elif i % 5 == 0:
        print("Buzz")
    else:
        print(i)
```

Another Big Problem with Word-Level Tokenization

Out-of-Vocabulary (OOV) Words

- ▶ What happens when the model sees a word at **test time** that never appeared in the training data?
 - ⦿ Cannot assign it an ID, and there is **no** embedding for it

Common workaround: the <UNK> token

- ▶ Replace rare words during training with <UNK>
- ▶ Use the same token for unseen words at test time

Problem with UNK: All unseen words collapse to the **same representation**.

- 1 Is “quantum” really the same as “aardvark”?
- 2 We don’t want to generate UNK when generating text (imagine ChatGPT doing this)

Other Limitations of Word-Level Tokenization

Morphological Explosion

Word-level tokenization treats different forms of the same word as **completely unrelated types**:

open, opened, opens, opening



separate IDs, separate embeddings

What we would like instead:

- ▶ Share statistical strength across related word forms
- ▶ Capture regularities like prefixes, suffixes, and stems

An Alternative: Character-Level Tokenization

idea: Tokenize text into individual characters instead of words.

- ▶ Very small vocabulary (just characters)
- ▶ No out-of-vocabulary (OOV) problem
- ▶ Naturally handles morphology and spelling variants

But... a serious downside

- ▶ Input sequences become much longer (models have a fixed token budget)

Problem LLMs have a fixed **token budget**.

the children are happy

Word tokens: [the, children, are, happy]

Character tokens: [t h e c h i l d r e n a r e h a p p y]

Another Major Problem with Character-Level Tokenization

- ▶ Characters are much less expressive than words
- ▶ Word-level structure is no longer given for free.
- ▶ Instead of “good”, the model sees “g o o d”

Key consequence:

- ▶ Character-level models must **discover that words exist**
 - ⊙ This structure is built-in for word-level tokenization

Character, Word, and Subword Tokenization

▶ Character-level

- ◉ Very small vocabulary
- ◉ But characters are weak units: learning a useful representation for “t” is much harder than for “today”

▶ Word-level

- ◉ Expressive tokens with clear semantics
- ◉ But very large vocabularies
- ◉ Out-of-vocabulary (OOV) words $\Rightarrow$ use of <UNK> token
- ◉ Data inefficiency: harder to learn good embeddings for rare words
- ◉ Large embedding matrices $\Rightarrow$ more memory and computation

▶ Subword-level (Hybrid)

- ◉ Middle ground between characters and words
- ◉ Reasonable vocabulary size
- ◉ Can represent unseen words by composing known subwords

Subword Tokenization (circa 2016)

Key idea: Represent text using **word pieces** rather than full words or characters.

- ▶ Frequent words are represented with **fewer tokens** (e.g., "the" is a single token)
- ▶ Rare or unseen words are decomposed into **multiple subwords** e.g., "unrelated" could be broken up into "un", "relate", "ed"
- ▶ Related words can **share parameters** (e.g., open, opened, opening)
- ▶ Smaller vocabulary → fewer parameters, less memory, faster computation

How subwords are learned: Most methods rely on simple frequency-based algorithms, such as **Byte Pair Encoding (BPE)**.

Work in Machine Translation (MT) led to subword tokenization

Later adopted by BERT, T5, RoBERTa, GPT, and others.

Neural Machine Translation of Rare Words with Subword Units

Rico Sennrich and **Barry Haddow** and **Alexandra Birch**

School of Informatics, University of Edinburgh

`{rico.sennrich,a.birch}@ed.ac.uk, bhaddow@inf.ed.ac.uk`

Machine Translation also led to strong sequence encoders, the attention-based models - Transformers!

Byte Pair Encoding (BPE): Step 0

Goal: Learn a set of subword units from raw text using frequency statistics.

Training data (toy example):

Word	Frequency
hug	10
pug	5
pun	12
bun	4
hugs	5

Initial vocabulary (characters only):

$\{ b, g, h, n, p, s, u \}$

We will now **iteratively merge the most frequent character pairs**.

Example adapted from Hugging Face documentation

Byte Pair Encoding (BPE): Step 1

Step 1: Count how often each **adjacent character pair** occurs in the training data.

Training data (with frequencies):

Word	Frequency
hug	10
pug	5
pun	12
bun	4
hugs	5

Character pair counts:

Pair	Total Frequency
u g	20
p u	17
u n	16
h u	15
g s	5

Result: most frequent pair is **u g** - it will be merged next.

Byte Pair Encoding (BPE): Step 2

Step 2: Merge the most frequent character pair **ug** into a new symbol, and retokenize the data.

After merging $u + g \rightarrow ug$:

Word	Frequency
$h + ug$	10
$p + ug$	5
$p + u + n$	12
$b + u + n$	4
$h + ug + s$	5

Updated vocabulary:

$$\{ b, g, h, n, p, s, u, \mathbf{ug} \}$$

Next: Recompute pair frequencies on the retokenized data, then repeat.

Byte Pair Encoding (BPE): Repeat

Step 3: Repeat the same procedure

- ▶ Recompute pair frequencies on the retokenized data
- ▶ Merge the most frequent adjacent pair
- ▶ Add the merged symbol to the vocabulary
- ▶ Retokenize all words

Example continuation:

- ▶ Next merge: $u + n \rightarrow un$
- ▶ Later merge: $h + ug \rightarrow hug$

Stop when a fixed vocabulary size is reached.

Byte Pair Encoding (BPE): Stopping

Stopping criterion: After a fixed number of merge operations (or once a target vocabulary size is reached), we stop.

Resulting tokenization (example):

Tokenized word	Frequency
hug	10
p + ug	5
p + un	12
b + un	4
hug + s	5

Final vocabulary:

$\{ b, g, h, n, p, s, u, ug, un, hug \}$

Algorithm 1 Learn BPE operations

```
import re, collections

def get_stats(vocab):
    pairs = collections.defaultdict(int)
    for word, freq in vocab.items():
        symbols = word.split()
        for i in range(len(symbols)-1):
            pairs[symbols[i], symbols[i+1]] += freq
    return pairs

def merge_vocab(pair, v_in):
    v_out = {}
    bigram = re.escape(' '.join(pair))
    p = re.compile(r'(?!\S)' + bigram + r'(?!\S)')
    for word in v_in:
        w_out = p.sub(' '.join(pair), word)
        v_out[w_out] = v_in[word]
    return v_out

vocab = {'l o w </w>' : 5, 'l o w e r </w>' : 2,
        'n e w e s t </w>':6, 'w i d e s t </w>':3}
num_merges = 10
for i in range(num_merges):
    pairs = get_stats(vocab)
    best = max(pairs, key=pairs.get)
    vocab = merge_vocab(best, vocab)
    print(best)
```

- ▶ Initialize vocab with characters
- ▶ While vocab size < max vocab size, do:
 - ◉ Count all adjacent symbol pairs
 - ◉ Merge the most frequent pair

BPE Tutorial & Implementation

- ▶ This tutorial walks through a full implementation step by step
 - ◉ Uses **byte-level BPE** (as in GPT-2), rather than character-level
 - ◉ Core idea is exactly the same: iteratively merge frequent pairs
- ▶ Video tutorial (Andrej Karpathy):
<https://www.youtube.com/watch?v=zduSFxRajkE>

Questions?